B. Battling with Numbers

time limit per test: 1 second
memory limit per test: 256 megabytes

On the trip to campus during the mid semester exam period, Chaneka thinks of two positive integers X and Y. Since the two integers can be very big, both are represented using their prime factorisations, such that:

- $X = A_1^{B_1} \times A_2^{B_2} \times \ldots \times A_N^{B_N}$ (each A_i is prime, each B_i is positive, and $A_1 < A_2 < \ldots < A_N$)
- $Y = C_1^{D_1} \times C_2^{D_2} \times \ldots \times C_M^{D_M}$ (each C_j is prime, each D_j is positive, and $C_1 < C_2 < \ldots < C_M$)

Chaneka ponders about these two integers for too long throughout the trip, so Chaneka's friend commands her "Gece, deh!" (move fast) in order to not be late for the exam.

Because of that command, Chaneka comes up with a problem, how many pairs of positive integers p and q such that $\text{LCM}(p, q) = X$ and $\text{GCD}(p, q) = Y$. Since the answer can be very big, output the answer modulo $998\,244\,353$.

Notes:

- $\text{LCM}(p, q)$ is the smallest positive integer that is simultaneously divisible by p and q.
- $\text{GCD}(p, q)$ is the biggest positive integer that simultaneously divides p and q.

Input

The first line contains a single integer N $(1 \leq N \leq 10^5)$ — the number of distinct primes in the prime factorisation of X.

The second line contains N integers $A_1, A_2, A_3, \ldots, A_N$ $(2 \leq A_1 < A_2 < \ldots < A_N \leq 2 \cdot 10^6$; each A_i is prime) — the primes in the prime factorisation of X.

The third line contains N integers $B_1, B_2, B_3, \ldots, B_N$ $(1 \leq B_i \leq 10^5)$ — the exponents in the prime factorisation of X.

The fourth line contains a single integer M $(1 \leq M \leq 10^5)$ — the number of distinct primes in the prime factorisation of Y.

The fifth line contains M integers $C_1, C_2, C_3, \ldots, C_M$ $(2 \leq C_1 < C_2 < \ldots < C_M \leq 2 \cdot 10^6$; each C_j is prime) — the primes in the prime factorisation of Y.

The sixth line contains M integers $D_1, D_2, D_3, \ldots, D_M$ $(1 \leq D_j \leq 10^5)$ — the exponents in the prime factorisation of Y.

Output

An integer representing the number of pairs of positive integers p and q such that $\text{LCM}(p, q) = X$ and $\text{GCD}(p, q) = Y$, modulo $998\,244\,353$.

Examples

input (Copy)

```
4
2 3 5 7
2 1 1 2
2
3 7
1 1
```

output (Copy)

```
8
```

Given

gcd(p,q) = Y
lcm(p,q) = X

Prime factorizations of X and Y

p · q = gcd(p,q) · lcm(p,q)

Necessary condition

-> Y must divide X, otherwise answer = 0

Prime-wise Analysis

For a prime r:

Let

exponent in X = B
exponent in Y = D
exponent in p = α
exponent in q = β

Constraints:

min(α, β) = D,   max(α, β) = B

Cases per prime

```
Condition | Ways
----------- | -----------------------------
( D > B ) | 0 (impossible)
          |
( D = B ) | 1 → ( (α, β) = (B,B) )
          |
( D < B ) | 2 → ( (B, D) ) or ( (D, B) )
```

Prime only in X ⇒ D = 0 < B ⇒ 2 ways

Prime only in Y ⇒ impossible ⇒ 0

Final Answer

$$\text{Answer} = \prod_{p \text{ prime}} \big(\text{ways per prime}(p)\big) \pmod{998244353}$$

Algorithm

1. Merge primes of X and Y

2. For each prime:

   - if in Y but not in X → print 0
   - compare exponents → multiply by 1 or 2

3. Output result

GCD fixes the minimum exponent, LCM fixes the maximum — each prime independently gives 0, 1, or 2 choices.

Code

```cpp
#include <bits/stdc++.h>
using namespace std;

static const long long MOD = 998244353;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N;
    cin >> N;
    vector<int> A(N), B(N);
    for (int i = 0; i < N; i++) cin >> A[i];
    for (int i = 0; i < N; i++) cin >> B[i];

    int M;
    cin >> M;
    vector<int> C(M), D(M);
    for (int i = 0; i < M; i++) cin >> C[i];
    for (int i = 0; i < M; i++) cin >> D[i];

    long long ans = 1;

    int i = 0, j = 0;
    while (i < N || j < M) {
        if (j == M || (i < N && A[i] < C[j])) {
            // Prime only in X
            // D = 0, B > 0 → contributes 2
            ans = (ans * 2) % MOD;
            i++;
        } else if (i == N || C[j] < A[i]) {
            // Prime in Y but not in X → impossible
            cout << 0 << "\n";
            return 0;
        } else {
            // Same prime
            int b = B[i];
            int d = D[j];
            if (d > b) {
                cout << 0 << "\n";
                return 0;
            }
            if (d < b) {
                ans = (ans * 2) % MOD;
            }
            // else d == b → multiply by 1
            i++;
            j++;
        }
    }

    cout << ans << "\n";
    return 0;
}
```